# Project Akhir Inkubasi DevOps POROS 2026

## CI/CD Pipeline & Monitoring

Project MODERNO Fashion & Apparel Ecommerce

---

| | |
|---|---|
| **Topik** | CI/CD Pipeline, Containerization, dan Observability |
| **Durasi** | 1 minggu (7 hari kalender) |
| **Bentuk** | Individu |
| **Pengumpulan** | Pengumpulan Tugas |
| **Penamaan File** | `NIM_NAMA_TUGAS_AKHIR_INKUBASI_DEVOPS.pdf` |

---

## Latar Belakang

Project **MODERNO** adalah aplikasi ecommerce fullstack yang terdiri dari:

| Komponen | Teknologi | Port |
|---|---|---|
| Frontend | Next.js 14, TypeScript, Tailwind CSS | 3000 |
| Backend API | NestJS, TypeORM, Passport JWT | 3001 |
| Database | PostgreSQL 15 | 5432 |

Arsitektur ini mencerminkan sistem production nyata yang membutuhkan pipeline deployment yang andal, cepat, dan dapat dipantau. Peserta diminta merancang dan mengimplementasikan infrastruktur DevOps lengkap di atas project MODERNO — mulai dari version control, CI/CD pipeline otomatis, containerization, hingga monitoring.

## Tugas 1 — Version Control & Branching Strategy

Setup repositori GitHub dengan strategi branching yang tepat untuk mendukung pengembangan paralel dan release management.

### Yang harus dikerjakan

1. Fork repositori GitHub MODERNO yang diberikan
2. Implementasikan branching strategy menggunakan salah satu model:
    - Git Flow (`main`, `develop`, `feature/*`, `release/*`, `hotfix/*`)
    - GitHub Flow (`main` + feature branches)
    - Trunk-based Development
3. Buat dokumentasi branching strategy dalam laporan PDF
4. Sertakan commit message convention — Conventional Commits direkomendasikan

---

# Tugas 2 — Containerization & Docker Optimization

Buat konfigurasi Docker dan pastikan semua service berjalan dengan benar di lingkungan containerized.

## Yang harus dikerjakan

1. Buat `Dockerfile` untuk `client` dan `server`:
    - Gunakan **multi-stage build** untuk meminimalkan image size
    - Tambahkan `.dockerignore` yang tepat
2. Buatlah `docker-compose.yml`:
    - Tambahkan health checks untuk semua service
    - Konfigurasi resource limits (CPU & memory)
    - Gunakan named volumes untuk persistensi data
    - Pisahkan `docker-compose.yml` (dev) dan `docker-compose.prod.yml`
3. Dokumentasikan cara menjalankan project secara lokal dengan Docker
4. Sertakan output `docker images ls` dan `docker stats` sebagai bukti

---

# Tugas 3 — CI Pipeline dengan GitHub Actions

Bangun Continuous Integration pipeline yang otomatis berjalan setiap ada push atau pull request ke repository.

## Stage 1 — Code Quality

- **Linting:** ESLint untuk frontend (Next.js) dan backend (NestJS)
- **Format check:** Prettier

## Stage 2 — Testing

- **Unit tests:** Jest untuk NestJS services dan controllers

## Stage 3 — Build

- Build Docker image untuk `client` dan `server`
- Tag image dengan: `latest`, branch name, dan commit SHA
- Push image ke registry (Docker Hub atau GitHub Container Registry)

## Konfigurasi wajib di semua stage

- Environment secrets management menggunakan GitHub Secrets

---

# Tugas 4 — CD Pipeline & Deployment

Implementasikan Continuous Deployment pipeline untuk men-deploy aplikasi MODERNO ke environment staging dan production.

### Yang harus dikerjakan

1. Buat dua environment deployment:
    - **Staging:** deploy otomatis dari branch `develop` setelah CI pass
    - **Production:** deploy manual dengan approval dari branch `main`
2. Pilih salah satu strategi deployment dan jelaskan alasannya:
    - Blue-Green Deployment
    - Rolling Update
    - Canary Deployment
3. Implementasikan **rollback otomatis** jika health check gagal setelah deployment
4. Konfigurasi environment variables yang berbeda untuk staging dan production menggunakan GitHub Environments
5. Buat **deployment runbook** dalam format Markdown yang mencakup:
    - Pre-deployment checklist
    - Deployment steps
    - Post-deployment verification
    - Rollback procedure
6. Implementasikan database migration sebagai bagian dari pipeline (gunakan TypeORM migrations)

---

## Tugas 5 — Monitoring & Observability

Setup monitoring Metrics untuk aplikasi MODERNO.

### 5.1 Metrics — Prometheus + Grafana

1. Expose metrics dari NestJS menggunakan `@willsoto/nestjs-prometheus`:
    - HTTP request rate, latency (p50, p95, p99)
    - Error rate per endpoint
    - Active connections
    - Database query duration
    - Custom business metrics: `order_created_total`, `cart_items_added_total`
2. Konfigurasi Prometheus scrape config untuk mengumpulkan metrics
3. Buat Grafana dashboard yang mencakup:
    - Overview panel: uptime, request rate, error rate
    - Latency heatmap
    - Top 5 slowest endpoints
    - Business metrics panel

---

## Deliverables

Kumpulkan semua item berikut dalam satu file PDF:

| No | Deliverable | Format | Keterangan |
|---|---|---|---|
| 1 | Link Repository GitHub | URL | Repository harus public |
| 2 | CI/CD Pipeline Config | `.yml` files | Semua file workflow GitHub Actions |
| 3 | Docker Configuration | Dockerfile + compose | Semua versi (dev, prod, override) |
| 5 | Monitoring Config | YAML + JSON | Prometheus, Grafana dashboard JSON, dll |
| 6 | Screenshot Bukti | PNG/PDF | Pipeline running, Grafana dashboard, dll |
| 7 | Penjelasan | | Sama seperti tugas-tugas sebelumnya setiap langkah harus diberikan penjelasan |

## Catatan Penting

### Diperbolehkan

- Menggunakan free tier cloud provider (AWS Free Tier, GCP Always Free, Railway, Render, Fly.io)
- Menggunakan GitHub Actions free tier (2000 menit/bulan untuk public repo)
- Menggunakan Docker Desktop untuk pengujian lokal
- Berdiskusi konsep dengan sesama peserta — namun implementasi harus mandiri

### Tidak Diperbolehkan

- Menyalin konfigurasi pipeline dari repository lain tanpa modifikasi dan pemahaman
- Meninggalkan credentials (API key, password) hardcoded dalam repository

### Tips

- Mulai dari Tugas 1 dan 2, lanjutkan secara berurutan
- Gunakan `docker-compose` untuk menjalankan monitoring stack secara lokal sebelum deploy ke cloud
- Manfaatkan GitHub Actions marketplace untuk action yang sudah tersedia
- Dokumentasikan setiap keputusan desain dalam laporan teknis

---

> Selamat mengerjakan! Pertanyaan teknis diajukan melalui wa Aldura.